

Attention Multihop Graph and Multiscale Convolutional Fusion Network for Hyperspectral Image Classification

Hao Zhou¹, Fulin Luo¹, *Senior Member, IEEE*, Huiping Zhuang, *Member, IEEE*, Zhenyu Weng, *Member, IEEE*, Xiuwen Gong², and Zhiping Lin¹, *Senior Member, IEEE*

Abstract—Convolutional neural networks (CNNs) for hyperspectral image (HSI) classification have generated good progress. Meanwhile, graph convolutional networks (GCNs) have also attracted considerable attention by using unlabeled data, broadly and explicitly exploiting correlations between adjacent parcels. However, the CNN with a fixed square convolution kernel is not flexible enough to deal with irregular patterns, while the GCN using the superpixel to reduce the number of nodes will lose the pixel-level features, and the features from the two networks are always partial. In this article, to make good use of the advantages of CNN and GCN, we propose a novel multiple feature fusion model termed attention multihop graph and multiscale convolutional fusion network (AMGCFN), which includes two subnetworks of multiscale fully CNN and multihop GCN to extract the multilevel information of HSI. Specifically, the multiscale fully CNN aims to comprehensively capture pixel-level features with different kernel sizes, and a multihead attention fusion module (MAFM) is used to fuse the multiscale pixel-level features. The multihop GCN systematically aggregates the multihop contextual information by applying multihop graphs on different layers to transform the relationships between nodes, and an MAFM is adopted to combine the multihop features. Finally, we design a cross-attention fusion module (CAFM) to adaptively fuse the features of two subnetworks. The AMGCFN makes full use of multiscale convolution and multihop graph features, which is conducive to the learning of multilevel contextual semantic features. Experimental results on three benchmark HSI datasets show that the AMGCFN has a better performance than a few state-of-the-art methods. Code: https://github.com/EdwardHaoz/IEEE_TGRS_AMGCFN.

Index Terms—Attention fusion, convolutional neural network (CNN), graph convolutional network (GCN), hyperspectral image (HSI), land-cover classification.

Manuscript received 20 January 2023; revised 20 March 2023; accepted 6 April 2023. Date of publication 10 April 2023; date of current version 21 April 2023. This work was supported in part by the National Natural Science Foundation of China under Grant 62071340 and Grant 61801336 and in part by the Natural Science Foundation of Chongqing under Grant CSTB2022NSCQ-MSX0452. (Corresponding author: Fulin Luo.)

Hao Zhou, Zhenyu Weng, and Zhiping Lin are with the School of Electrical and Electronic Engineering, Nanyang Technological University, Singapore 639798 (e-mail: hzhou009@e.ntu.edu.sg; zhenyu.weng@ntu.edu.sg; ezplin@ntu.edu.sg).

Fulin Luo is with the College of Computer Science, Chongqing University, Chongqing 400044, China (e-mail: luoflyn@163.com).

Huiping Zhuang is with the Shien-Ming Wu School of Intelligent Engineering, South China University of Technology, Guangdong 510641, China (e-mail: hpzhuang@scut.edu.cn).

Xiuwen Gong is with the Faculty of Engineering, The University of Sydney, Sydney, NSW 2000, Australia (e-mail: xiuwen.gong@sydney.edu.au).

Digital Object Identifier 10.1109/TGRS.2023.3265879

I. INTRODUCTION

HYPERSPECTRAL images (HSIs) are composed of hundreds of continuous wavelength bands, which contains rich spectral and spatial information, and are widely used in various fields, including geological exploration, environmental monitoring, agricultural assessment, and urban planning [1], [2]. These applications are based on the accurate prediction of HSI per pixel, so in the field of remote sensing, the task of HSI classification has always been a hot research topic [3], [4].

For HSI classification, the difficulties of HSI classification different from traditional image classification problems were revealed. The rich spectral information makes it difficult for the classifier to obtain accurate classification results [5]. In recent years, deep learning with better ability to capture features by deep networks has brought new ideas for HSI classification. Deep learning has shown great potential and generality in dealing with a series related fields of signal processing, such as natural language processing (NLP) and computer vision (CV) [6], [7]. As the most commonly used deep learning model [8], [9], convolutional neural network (CNN) can better overcome the influence of spectral noise, spatial variability of spectral features, and other ambiguities in the data by combining the spatial context [10]. CNN extracts spatial information through local connection weight sharing, which significantly reduces the number of parameters compared with fully connected networks, such as stacked autoencoder (SAE) [11], and CNNs have been widely applied to HSI classification [12]. Subsequently, a series of variants from 1-D CNN to 3-D CNN and from single CNN to hybrid CNN are derived to enhance the spectral-spatial feature representation performance [13], [14]. Xie et al. [15] proposed a semantic and spatial-spectral feature fusion transformer (S3FFT) network for obtaining spatial-spectral features of HSI. Multiscale feature representation is another important variant of CNN. Yang et al. [16] tried to extract and fuse multiscale features. A two-stream CNN [17] was proposed to extract spectral features as well as local and global spatial features through a 2-D CNN. Similarly, Hao et al. [18] also used two-stream CNN for HSI classification. ResNet [19] and DenseNet [20] are also introduced into HSI classification for increasing network depth. In addition, the attention mechanism [21] is also integrated into the neural network

and applied to HSI classification, such as the dual-branch dual-attention (DBDA) network [22] and hierarchical residual network with attention mechanism (HResNetAM) [23].

Although the abovementioned iterative neural network model based on the backpropagation supervised learning method has achieved considerable results, the CNN model still has considerable limitations. The convolution based on Euclidean data is difficult to grasp the complex topological structure, so it is highly dependent on the land cover of short-range space, and needs to train and learn many network parameters. At the same time, CNN also needs to contain a large number of manually labeled samples to form a training set. Liu et al. [24] proposed a deep contrast learning network (DCLN) method for small sample HSI. To this end, unsupervised and semisupervised learning methods have gradually attracted attention, because they can obtain information from a large number of unlabeled samples, such as a residual-based unsupervised convolutional network [25]. Graph-based semisupervised learning [26], [27] can make full use of the unlabeled samples by exploiting the graph structure of the data. The graph convolutional network (GCN) [28] generalizes the convolution idea to graphs of arbitrary structure and has successfully achieved results in various NLP tasks, which can also be applied to HSI classification, such as nonlocal GCN [29]. Due to the large number of HSI pixels, it is difficult to construct a graph with a single pixel as a node. The method of encoding HSI into a graph usually adopts superpixel segmentation. Wan et al. [30] proposed to aggregate pixels with close spatial distances into superpixel nodes according to spectral similarity, which greatly reduces the number of nodes in the graph. This also makes the adaptively learned superpixel nodes break through the limitations of the original Euclidean data structure and can more clearly define the correlation between adjacent irregular areas, thereby simulating the spatial context structure of HSI. The studies of Ding et al. [31], [32] are also based on GCN to explore the HSI classification problem. The attention mechanism is also applied in GCN, such as the graph attention network (GAT) [33] model that dynamically changes the weight between nodes.

Both CNN and GCN are deep learning methods that can capture deep features, focusing on pixel-level features and superpixel-level features, respectively. So, how to design and implement a fusion scheme to integrate the HSI classification ability is worthy of attention. However, the hybrid network constructed by directly combining the existing schemes has the problem of an incompatible data structure. The features obtained by CNN cannot be directly fused with the features extracted by GCN, which are obtained by constructing the graph structure through superpixel segmentation. In the subsequent fusion, continuous data conversion is required, which seriously increases the training time. Therefore, a direct fusion network of multiscale CNN and GCN should be constructed to match the characteristics of HSI. In addition, the fusion network should also consider how to balance the CNN and GCN subnetworks in training, because incomplete training of either subnetwork will reduce the classification performance. In response to this problem,

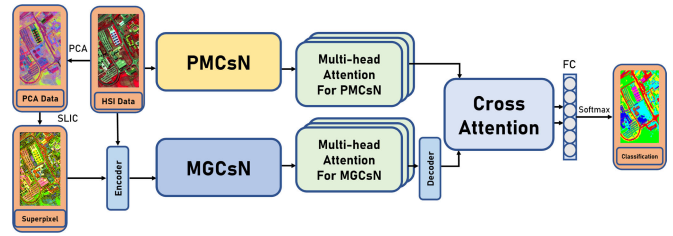


Fig. 1. Architecture of the proposed AMGCFN.

Liu et al. [34] proposed CNN-enhanced GCN (CEGCN), which integrated CNN and GCN into a single network by encoding and decoding graph structures. Ding et al. [35] also explored hybrid networks. The weighted feature fusion network (WFCG) [36] designed a network with two branches, extracting superpixel features based on GAT and passing them to CNN for combination to realize the capture of long-distance depth features.

Whether it is CNN or GCN, the network constructed by only a single convolutional layer has limited ability to capture information, which is the limitation of the receptive field. CNN-based networks usually use the method of intercepting surrounding pixels centered on the sampling point to construct a patch, and apply CNN to this small neighborhood area. Such methods can only obtain patches of fixed size, ignoring the scale information of HSI. In HSI, the scales of different features are different, and the use of local patches of the same size will inevitably lead to poor classification of some categories of features. For GCN, the network learns on a predefined single graph structure, and superpixel nodes can only aggregate the information of the nearest neighbor nodes. For this reason, Gao et al. [37] proposed a multiscale residual network (MSRN), which replaces the ordinary convolution with a mixed depth convolution (MDConv) containing multiple size kernels to capture multiscale information. Liu et al. [38] proposed multilevel superpixel structured graph U-Net (MSSGU) to extract multiscale information by gradually merging adjacent superpixels and converting them into multilevel graphs. Wan et al. [39] proposed a dynamic graph convolution model that simultaneously refines graph edge weights and relationships between image regions by learning an improved similarity measure and an “edge filter.” An enlightening solution for improving GCN is to reconstruct the adjacency matrix of nodes, but changing the connection of nodes may introduce redundant information and reduce the expressiveness of the network. Xue et al. [40] proposed a multihop GCN. The model has multiple branches, and each branch extracts node representations from a k -hop graph with small kernels. Such a design systematically aggregates multiscale contextual information without adding redundant information.

The above attempts to break through the limitations of the receptive field to extract multiscale information often exist in a single type of convolutional network, and focus on one of CNN or GCN. Therefore, how to increase the ability to extract multiscale information while retaining the advantages of the hybrid network for both pixel-level and super-pixel-level information, and fully integrate the

information extracted by the two networks is the main motivation for us. We propose an attention multihop graph and multiscale convolutional fusion network (AMGCFN). As shown in Fig. 1, the proposed AMGCFN consists of a parallel multiscale convolutional subnetwork (PMCsN), a multihop graph convolutional subnetwork (MGCsN), two multihead attention fusion modules (MAFMs), and a cross-attention fusion module (CAFM). PMCsN extracts multiscale pixel-level information through 2-DCNN with different size kernels in parallel multiple paths, and MAFM is used to fuse the multiscale features. MGCsN pregenerates multihop graph long-range aggregation to extract superpixel-level information according to the adjacency matrix, and the multihop features are combined by MAFM. Finally, the two subnetworks pass through CAFM to obtain the discriminative features for classification. Due to the multiscale and multihop blessing, AMGCFN has comprehensive and multilevel capture capabilities for the spectral-spatial depth features of HSI. The main contributions are summarized as follows.

- 1) Aiming at the fixed kernel problem of traditional CNN layers, we introduce a multiscale kernel to build a parallel structure, capture pixel-level features from multiple scales, add cross-path connections to the multiscale network, and design the multipath feature fusion method based on multihead attention to promote feature fusion and interoperability.
- 2) For the narrow receptive field of traditional GCNs, we propose the multihop graphs to extract superpixel-level features from large-region neighbor nodes. A cascaded network is formed by stacking multihop GCNs to effectively extract the structural information. A multihead attention module is designed to combine the multihop features, and the residual error is introduced to enhance the robustness of network.
- 3) To balance and promote the deep fusion of subnetwork features, we design a CAFM to highlight the significant features and enhance the feature fusion of different subnetworks. This module extracts the prominent identification information to obtain more accurate classification results.
- 4) A hybrid-enhanced feature extraction framework, AMGCFN, is proposed based on PMCsN, MGCsN, MAFM, and CAFM. AMGCFN has a strong ability to capture multilevel features. Through in-depth experiments on three datasets, the effectiveness of the model is verified.

The rest of this article is organized as follows. The proposed AMGCFN is introduced in Section II. The experiments are described in Section III. Corresponding discussion and analysis are carried out in Section IV. The conclusions are presented in Section V.

II. PROPOSED METHOD

We denote HSI as $X \in \mathbb{R}^{H \times W \times B}$, and H, W , and B denote the height, width, and number of bands. The goal is to predict labels for all pixels in HSI based on limited labeled data. Our AMGCFN is a hybrid neural network that includes superpixel and pixel feature extraction and fusion to represent

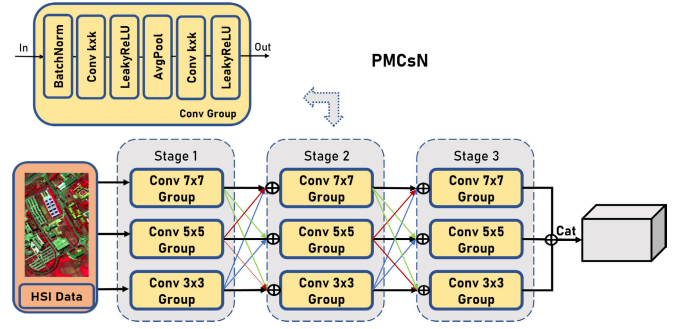


Fig. 2. Architecture of the proposed PMCsN.

the spectral and spatial features of HSI. The overall structure of the proposed AMGCFN is shown in Fig. 1, which consists of three main components: 1) PMCsN; 2) MGCsN; 3) MAFM; and 4) CAFM.

A. PMCsN

The 2-D CNN [41] is widely used in the field of natural image processing due to its powerful spatial context feature extraction ability, and the formula is

$$T^l(x, y) = \sigma \left(\sum_{h=0}^{H-1} \sum_{w=0}^{W-1} s^l(h, w) T^{l-1}(x+h, y+w) + b^l \right) \quad (1)$$

where $T^l(x, y)$ represents the output data of the l th layer at the position (x, y) , $T^{l-1}(x+h, y+w)$ represents the specific value of the feature map at the $(x+h, y+w)$ position in the $(l-1)$ th layer, s^l represents the specific value of the convolution kernel of the l layer at the position (h, w) , H and W represent the height and width of the convolution kernel, respectively, b^l represents the bias of the l th layer, and $\sigma(\cdot)$ represents the activation function; in PMCsN, we all use LeakyReLU.

For the input data, we adjust the distribution by batch normalization (BN) at each convolutional layer unit to prevent overfitting and speed up network training. The BN formula is as follows:

$$\hat{T}^l = \frac{T^l - E(T^l)}{\text{Var}(T^l)} \quad (2)$$

where T^l is the input of the l th layer and $\hat{T}^l$ is the normalized result of T^l after passing through the BN layer. $E(\cdot)$ and $\text{Var}(\cdot)$ represent the mean and variance function, respectively.

To enhance the feature collection capability of CNN-based networks and break through the limitation of the receptive field of a single fixed kernel, we design a parallel multiscale information collection network containing kernels of various sizes. Fig. 2 shows the structure of a PMCsN with three parallel paths divided into three stages. The kernel sizes were $K = 7 \times 7$, 5×5 , and 3×3 . However, due to the large spectral dimensions of HSI, directly applying 2-D CNN with different kernels to HSI will lead to too many parameters of the convolution kernel, which may easily lead to overfitting and reduce the network classification performance. Therefore, in PMCsN, we use two independent convolutional layers, consisting of a 1×1 kernel and a $k \times k$ kernel 2-D convolutional layer. The hidden layer feature obtained after

1×1 CNN is expressed as follows:

$$\mathbf{M}^l(x, y) = \sigma(\mathbf{m}^l \hat{\mathbf{T}}^{l-1}(x, y) + \mathbf{p}^l) \quad (3)$$

where $\mathbf{M}^l(x, y)$ represents the output data of the l th hidden layer unit at the position (x, y) , $\hat{\mathbf{T}}^{l-1}$ represents the specific value of the feature map of the $(l-1)$ th hidden layer unit at the (x, y) position, $\mathbf{m}^l$ represents the value of the convolution kernel of the l th hidden layer unit, $\mathbf{p}^l$ represents the bias of the l th hidden layer, and $\sigma(\cdot)$ represents the activation function. By transforming HSI with a 1×1 convolution kernel, the features can be extracted with fewer parameters. According to (1) and (3), the formula for obtaining the convolution structure is shown as follows:

$$\mathbf{T}^l(x, y) = \sigma \left(\sum_{h=0}^{K-1} \sum_{w=0}^{K-1} s^l(h, w) \mathbf{M}^l(x+h, y+w) + \mathbf{b}^l \right). \quad (4)$$

To reduce redundant image information and prevent overfitting, we add a layer of average pooling layer between the two layers of convolution and use the statistical information of subblocks in a certain area to describe the spatial distribution of all pixels in the area. Here, we adopt the average pooling to calculate the average value of all pixels in the area subblock. The pooling layer uses a pooling window of size 3×3 , the stride of each movement is 1, and the padding size is 1 to keep the image size unchanged.

In PMCsN, for the input of the next convolutional layer, we integrate the features extracted by different scale convolution kernels through cross-path fusion. The traditional multipath multiscale CNN network is relatively independent for each branch, and the features captured by each are integrated at the end. There is a gap between large-scale information and small-scale information, which limits the feature fusion ability of the network. In contrast, we repeatedly introduce cross-path connections among the three paths, so that the convolution module that captures large-scale information and small-scale information can be fully integrated before entering the next layer, for input $\mathbf{T}^{l-1}$

$$\mathbf{T}^{l-1} = \mathbf{T}_1^{l-1} + \mathbf{T}_2^{l-1} + \mathbf{T}_3^{l-1} \quad (5)$$

where $\mathbf{T}_1^{l-1}$, $\mathbf{T}_2^{l-1}$, and $\mathbf{T}_3^{l-1}$ represent the features from convolution kernel sizes $k = 7 \times 7, 5 \times 5$, and 3×3 .

Due to the different sizes of the convolution kernels, the padding operation is used to match the spatial resolution of the features, which is convenient for cross-path fusion and output after feature extraction. Through cross-path connection, the large-scale semantic information, the meso-scale spatial information, and the small-scale accurate information can be integrated with each other. The top-level information guides the feature capture of the smaller-level paths, and the smaller-level information supplements the semantic roughness of the top-level paths.

After completing the three-stage convolution through PMCsN, we splice the features, $\mathbf{T}_1$, $\mathbf{T}_2$, and $\mathbf{T}_3$, obtained by the three branches in the channel dimension for further fusion in subsequent modules

$$\mathbf{T}_{\text{out}} = \mathbf{T}_1 \oplus \mathbf{T}_2 \oplus \mathbf{T}_3 \quad (6)$$

where $\oplus$ denotes the concatenation operation.

B. MGCsN

As the input to GCN, we need to build the HSI into a graph structure. A traditional approach is to use each pixel in the HSI as a node, but this operation will make the number of nodes in the graph too large, and the subsequent computation through the adjacency matrix will be too large. To address this issue, we employ the simple linear iterative clustering (SLIC) for superpixel segmentation, which uses k -means clustering to generate superpixel graph nodes. Since the data structure of GCN needs to be fused with features from CNN in subsequent fusion modules, inspired by Liu et al. [34], we apply data structure transformation operations in the network.

At the HSI preprocessing stage, we construct the graph by superpixel segmentations and obtain the transformation matrix $\mathbf{Q} \in \mathbb{R}^{HW \times Z}$ between pixels and superpixels, where Z denotes the number of superpixels

$$\mathbf{Q}_{i,j} = \begin{cases} 1, & \text{if } \hat{\mathbf{X}}_i \in S_j \\ 0, & \text{if } \hat{\mathbf{X}}_i \notin S_j \end{cases} \quad \hat{\mathbf{X}} = \text{Flatten}(\mathbf{X}) \quad (7)$$

where $\hat{\mathbf{X}} \in \mathbb{R}^{HW \times B}$ is the shaping of the HSI original pixels, and the two spatial dimensions are flattened to combine a matrix, so $\mathbf{Q}_{i,j}$ reflects the relationships between the i th pixel element $\hat{\mathbf{X}}_i$ and the j th superpixel element S_j . Then, we can use the following formula to achieve the transformation between the nodes of graph and the superpixels as follows:

$$\mathbf{H} = \text{Encoder}(\mathbf{X}; \mathbf{Q}) = \bar{\mathbf{Q}}^T \text{Flatten}(\mathbf{X}) \quad (8)$$

$$\tilde{\mathbf{X}} = \text{Decoder}(\mathbf{H}; \mathbf{Q}) = \text{Reshape}(\mathbf{QH}) \quad (9)$$

where $\bar{\mathbf{Q}}$ denotes column-normalized $\mathbf{Q}$, $\mathbf{H}$ represents a node composed of superpixels, and $\text{Reshape}(\cdot)$ represents the spatial dimension of the restored flattened data. $\text{Encoder}(\cdot)$ means convert the pixmap to a graph node, and $\text{Decoder}(\cdot)$ represents the mapping of superpixel nodes to pixels.

MGCsN operates based on the undirected graph structure $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ obtained after superpixel segmentation, $\mathcal{V}$ represents the vertex set of $|\mathcal{V}| = N$, $\mathcal{E}$ is the edge set of the graph, $\mathcal{V}$ and $\mathcal{E}$ are encoded into the node matrix $\mathbf{H}$ and the adjacency matrix $\mathbf{A}$, and the normalized graph Laplacian matrix is defined as follows:

$$\mathbf{L} = \mathbf{I} - \mathbf{D}^{-\frac{1}{2}} \mathbf{A} \mathbf{D}^{-\frac{1}{2}} \quad (10)$$

where $\mathbf{I}$ is an identity matrix and $\mathbf{D} \in \mathbb{R}^{N \times N}$ is the degree matrix of the graph; i.e., $\mathbf{D}_{ii} = \sum_j \mathbf{A}_{ij}$.

For a graph signal $\mathbf{x} \in \mathbb{R}$ consisting of graph node eigenvectors, give a diagonal matrix filter $\mathbf{g}_\theta = \text{diag}(\theta)$, where $\theta \in \mathbb{R}^N$ are learnable parameters in the Fourier domain. Then, the graph convolution operator can be written as follows:

$$\mathbf{g}_\theta \star \mathbf{x} = \mathbf{U} \mathbf{g}_\theta \mathbf{U}^T \mathbf{x} \quad (11)$$

where $\mathbf{U}$ is the matrix of eigenvectors of $\mathbf{L}$; i.e., $\mathbf{L} = \mathbf{U} \mathbf{\Lambda} \mathbf{U}^T$, and $\mathbf{\Lambda}$ is the diagonal matrix of eigenvalues of $\mathbf{L}$.

It should be noted that the direct convolution calculation requires the Laplacian eigenvector, which is too computationally complex to implement for large graphs. To circumvent this obstacle, one possible way is to approximate the filter to

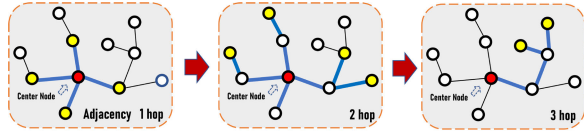


Fig. 3. Schematic of the multihop graph structure.

the order of k Chebyshev polynomials, such as K -localized convolution [42]

$$\mathbf{g}_\theta \star \mathbf{x} \approx \sum_{k=0}^K \theta'_k \mathbf{T}_k(\tilde{\mathbf{A}}) \mathbf{x} \quad (12)$$

where $\tilde{\mathbf{A}} = (2/\lambda_{\max})\mathbf{A} - \mathbf{I}$ with $\lambda_{\max}$ being the largest eigenvalue of $\mathbf{L}$. The Chebyshev polynomials are defined recursively by $\mathbf{T}_k(x) = 2x\mathbf{T}_{k-1}(x) - \mathbf{T}_{k-2}(x)$ with $\mathbf{T}_0(x) = 1$ and $\mathbf{T}_1(x) = x$.

Recently, Kipf and Welling [28] proposed to give a simplified version of (12) by restricting $k = 1$ and further approximate $\lambda_{\max} \approx 2$; this formula can be rewritten as follows:

$$\mathbf{g}_\theta \star \mathbf{x} \approx \theta \left(\mathbf{I} + \mathbf{D}^{-\frac{1}{2}} \mathbf{A} \mathbf{D}^{-\frac{1}{2}} \right) \mathbf{x}. \quad (13)$$

To avoid numerical instability and gradient explosion or disappearance caused by stacking of convolutional layers, $\mathbf{I} + \mathbf{D}^{-(1/2)} \mathbf{A} \mathbf{D}^{-(1/2)}$ is replaced by $\tilde{\mathbf{D}}^{-(1/2)} \tilde{\mathbf{A}} \tilde{\mathbf{D}}^{-(1/2)}$, where $\tilde{\mathbf{A}} = \mathbf{I} + \mathbf{D}$ and $\tilde{\mathbf{D}}_{ii} = \sum_j \tilde{\mathbf{A}}_{ij}$. As a result, the convolution operation of GCN model can be expressed as follows:

$$\mathbf{H}^l = \sigma \left(\tilde{\mathbf{D}}^{-\frac{1}{2}} \tilde{\mathbf{A}} \tilde{\mathbf{D}}^{-\frac{1}{2}} \mathbf{H}^{l-1} \mathbf{W}^l \right) \quad (14)$$

where $\mathbf{H}^l$ denotes the output of the l th layer and $\sigma(\cdot)$ represents the activation function, such as the ReLU function used in MGCsN. In addition, each convolution module in MGCsN contains one BN layer and two GCN layers, so the graph convolution operation is as follows:

$$\mathbf{H}^l = \sigma \left(\tilde{\mathbf{D}}^{-\frac{1}{2}} \tilde{\mathbf{A}} \tilde{\mathbf{D}}^{-\frac{1}{2}} \sigma \left(\tilde{\mathbf{D}}^{-\frac{1}{2}} \tilde{\mathbf{A}} \tilde{\mathbf{D}}^{-\frac{1}{2}} \hat{\mathbf{H}}^{l-1} \mathbf{W}_1^l \right) \mathbf{W}_2^l \right) \quad (15)$$

where $\hat{\mathbf{H}}^{l-1}$ represents the normalization of input $\mathbf{H}^{l-1}$ according to (2), and $\mathbf{W}_1^l$ and $\mathbf{W}_2^l$ represent the weight matrix of the two hidden GCN layers, respectively.

The GCN is usually stacked using multiple convolutional layers with fixed kernels. For single-layer GCN, it can be seen that nodes aggregate features and update information from the nearest neighbor nodes through the adjacency matrix $\mathbf{A}$ at the GCN layer. However, to capture long-range information, the enhanced GCN is designed by increasing the number of network layers and expanding the receptive field, which significantly increases the complexity of convolution operations and the number of parameters. This will cause the overfitting to reduce network performance. Therefore, the multihop graphs are used to construct the graph convolution and improve the discriminative performance of features.

Specifically, we use the segmentation map obtained by the SLIC algorithm to calculate the weight relationship between superpixel nodes through the Gaussian kernel function to

obtain the basic weight matrix $\mathbf{W}$, and the formula is

$$\mathbf{W}(i, j) = \begin{cases} e^{-\frac{\|\mathbf{S}_i - \mathbf{S}_j\|^2}{2\sigma^2}}, & \mathbf{S}_j \in \mathfrak{N}(\mathbf{S}_i) \\ 0, & \text{otherwise} \end{cases} \quad (16)$$

where $\mathfrak{N}(\mathbf{S}_j)$ is the spatial neighbor superpixel of $\mathbf{S}_i$. $\|\mathbf{S}_i - \mathbf{S}_j\|^2$ represents the calculation of the Euclidean distance between nodes. As the distance between two vectors increases, the value of the Gaussian kernel function decreases monotonically.

According to $\mathbf{W}$, we construct the adjacency matrix of multihop graph structure. For each superpixel node n , we use depth-first search (DFS) algorithm to find and record all paths that are k hops away from the selected center node, and the endpoint of the path is denoted as n_k . By calculating the mean value of node n and n_k path weights, we can get the k -hop graph adjacency matrix $\mathbf{A}_k$

$$\mathbf{A}_k(n, n_k) = \frac{1}{k} (\mathbf{W}(n, n_1) + \mathbf{W}(n_1, n_2) + \cdots + \mathbf{W}(n_{k-1}, n_k)) \quad (17)$$

where $n_1, n_2, \dots, n_{k-1}$ represent the intermediate nodes of the path. If there are multiple different paths between n and n_k , we retain the maximum weight.

Based on the above method, we can generate a series of k -hop graph adjacency matrices with weights $\mathbf{A}_1, \mathbf{A}_2, \dots, \mathbf{A}_k$. The multihop graph structure is shown in Fig. 3, where the neighbors of a center node are used as the one-hop graph, and the neighbors of neighbors of a center node are considered as the two-hop graph. For the one-hop graph, it is the same as the basic weight matrix. Such k -hop graph avoids introducing redundancy and, at the same time, expands the receptive field of a single node, making it connect with more nodes across neighboring nodes.

In the MGCsN structure shown in Fig. 4, we build a cascaded network based on the GCN layer and multihop graph, specifically from the one-hop adjacency matrix $\mathbf{A}_1$, to the two-hop adjacency matrix $\mathbf{A}_2$, as the network goes deeper into the k -hop adjacency matrix $\mathbf{A}_k$ to aggregate the node information. Such structures constitute the feature extraction pyramid. The extracted features of each layer are based on its previous layer and also include the previous neighbor node information. The input data $\mathbf{H}_{\text{in}}$ are first passed through a fully connected layer to unify the number of channels in the intermediate layer

$$\mathbf{H} = \sigma(\mathbf{W}_{\text{in}} \mathbf{H}_{\text{in}} + \mathbf{B}_{\text{in}}) \quad (18)$$

where $\mathbf{W}_{\text{in}}$ and $\mathbf{B}_{\text{in}}$ represent the weight and bias of the fully connected layer. In addition, since the GCN layers do not change the data shape of the feature, we add a residual structure to the cascade to enhance the robustness. The graph convolution operation of the model can be rewritten as follows:

$$\mathbf{H}^l = \hat{\mathbf{H}}^{l-1} + \sigma \left(\mathbf{D}^{-\frac{1}{2}} \tilde{\mathbf{A}}_s \mathbf{D}^{-\frac{1}{2}} \sigma \left(\mathbf{D}^{-\frac{1}{2}} \tilde{\mathbf{A}}_s \mathbf{D}^{-\frac{1}{2}} \hat{\mathbf{H}}^{l-1} \mathbf{W}_1^l \right) \mathbf{W}_2^l \right) \quad (19)$$

where $s = 1, 2, \dots, k$.

Different hops can perceive different visual field information, so using the output of each layer of MGCsN for

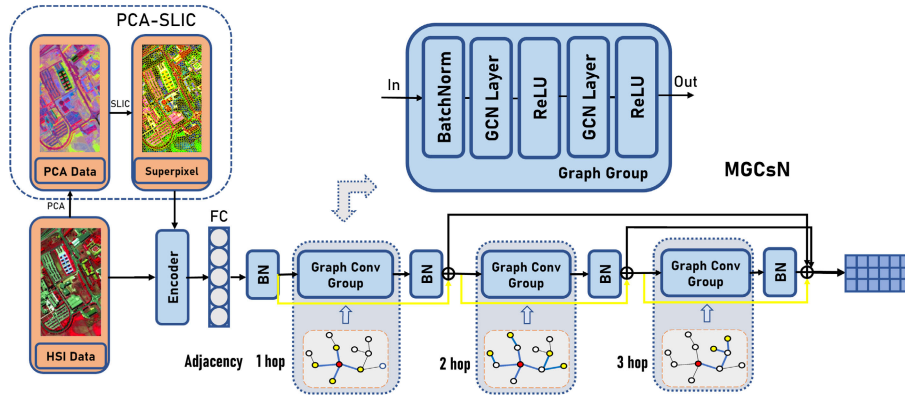


Fig. 4. Architecture of the proposed MGCsN.

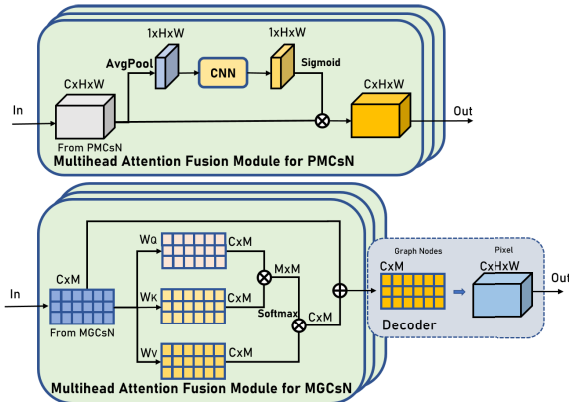


Fig. 5. Architecture of the MAFM.

multihead attention is beneficial to fuse the features of different hops. We splice the features (H^1 , H^2 , and H^3) obtained from different depths of the MGCsN in the channel dimension for the subsequent module

$$H_{\text{out}} = H^1 \oplus H^2 \oplus H^3. \quad (20)$$

C. Feature Fusion and Classification

To fuse the features of multiscale and multihop, we design two feature fusion modules based on multihead attention. To combine two subnetworks, a cross attention module is developed to obtain the final fusion features.

1) *Multihead Attention Fusion Module*: Attention brings many benefits to the HSI classification task. It can focus on the key features from the rich spectral-spatial information of HSI. Thus, more important information can occupy the main position when predicting the classification results, thereby improving the performance of the network. The multihead attention mechanism splits the parameter matrix into multiple subsections, as shown in Fig. 5. The overall size of the matrix remains unchanged, but the dimension corresponding to each head is changed. This allows the matrix to learn various information, while the computational cost is similar to that of a single head.

For the feature T extracted from PMCsN by (6) with shape $C \times H \times W$, we adopt an adaptive method based on the spatial attention mechanism for fusion. The specific method is to calculate the average value of the feature matrix to be fused

in the channel dimension, then obtain the weight coefficient after convolution and activation function, and, finally, multiply the original feature to obtain the weighted features. To better perform feature fusion, we use the multihead method to divide the input features into multiple heads on average in the channel dimension, and perform feature fusion on each subhead. The formula is

$$T_{\text{out}}^i = \text{Sigmoid}(f^{k \times k}(\text{AvgPool}(T^i))) \cdot T^i \quad (21)$$

where T^i represents the feature of the subhead, $\text{AvgPool}(\cdot)$ represents the average pooling of the channel dimension, $f^{k \times k}(\cdot)$ represents the convolution layer with the kernel of $k \times k$, we set to $k = 5$, and $\text{Sigmoid}(\cdot)$ indicates the activation function. After attention fusion for each subhead, we reassemble the multihead features

$$T_{\text{out}} = T_{\text{out}}^1 \oplus T_{\text{out}}^2 \oplus \dots \oplus T_{\text{out}}^n, \quad n = 1, 2, \dots \quad (22)$$

where n represents the number of subheads.

For the feature H obtained from MGCsN by (20), the shape is $C \times N$, and N is the number of superpixel nodes. Similar to PMCsN, we also use the multihead attention for fusion, which means that the input features are evenly cut in the channel dimension to obtain multiple subheads as self-attention units. Specifically, we multiply the input features with three weight matrices W^q , W^k , and W^v to get Q (query), K (key), and V (value), where $q_i = W^q h_i$, $k_i = W^k h_i$, and $v_i = W^v h_i$. The channel dimension of a matrix is d_k , and the self-attention score calculation formula can be written as follows:

$$H_{\text{out}}^i = \text{Attention}(H^i) + H^i = \text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V + H^i. \quad (23)$$

For the attention features of multihead, we concatenate them in the channel dimension

$$H_{\text{out}} = H_{\text{out}}^1 \oplus H_{\text{out}}^2 \oplus \dots \oplus H_{\text{out}}^n, \quad n = 1, 2, \dots \quad (24)$$

The features after multihead attention processing still retain the graph structure and need to map regions to pixels. The graph structure features can be correspondingly restored to the pixel form by (9), which is convenient for fusion with the features from PMCsN.

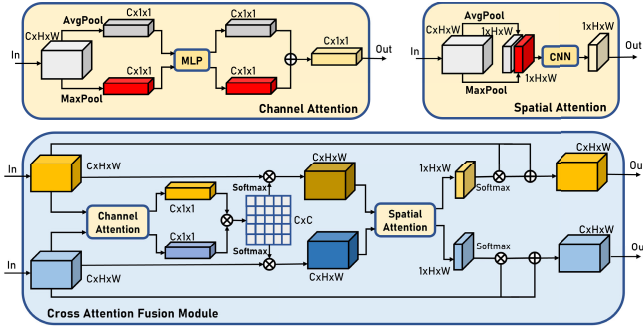


Fig. 6. Architecture of the CAFM.

2) *Cross-Attention Fusion Module*: According to (22) and (24), we obtain feature information extracted from PMCsN and MGCsN, but since the two branches are structurally independent of each other, inspired by Woo et al. [43] and Hou et al. [44], we design a cross-attention module for feature fusion. As shown in Fig. 6, our CAFM consists of two parts: channel attention cross module and spatial attention fusion module.

For the features F^c of shape $C \times H \times W$ from two branches, we perform the global maximum pooling and the average pooling to obtain two channel descriptions of shape $C \times 1 \times 1$. Then, they are input into a two-layer neural network, the number of neurons in the first layer is C/r , r is the scaling ratio, the activation function is ReLU, and the number of neurons in the second layer is C . This two-layer neural network is shared. Then, the obtained two features are added and passed the activation function to obtain the channel weight coefficients M^c . The formula is

$$\begin{aligned} M^c &= \sigma(\text{MLP}(\text{AvgPool}(F^c)) + \text{MLP}(\text{MaxPool}(F^c))) \\ &= \sigma(W_1(W_0(F_{\text{avg}}^c)) + W_1(W_0(F_{\text{max}}^c))) \end{aligned} \quad (25)$$

where W_1 and W_0 are the learnable parameters.

The channel weights M_T^c and M_H^c of the features T_{out} and H_{out} from the two subnetworks are obtained by the above method, and they are multiplied to obtain the cross matrix $M_{\text{cross}} = M_T^c(M_H^c)^T$ with shape $C \times C$. The features after channel attention cross module are shown as follows:

$$\begin{aligned} T^c &= \text{Softmax}(M_{\text{cross}})T_{\text{out}} \\ H^c &= \text{Softmax}(M_{\text{cross}}^T)H_{\text{out}}. \end{aligned} \quad (26)$$

After this, we perform spatial attention fusion on the two fused features. For the features F^s with a shape of $C \times H \times W$, we obtain two $1 \times H \times W$ spatial descriptions through the maximum pooling and the average pooling and concatenate these two descriptions in the channel dimension. Then, the weight coefficients M^s are obtained through a shared weight convolution layer with a kernel size of $k \times k$

$$\begin{aligned} M^s &= f^{k \times k}([\text{AvgPool}(F^s) \oplus \text{MaxPool}(F^s)]) \\ &= f^{k \times k}\left(\begin{bmatrix} F_{\text{avg}}^s \oplus F_{\text{max}}^s \end{bmatrix}\right) \end{aligned} \quad (27)$$

where $f^{k \times k}$ represents a convolutional layer with the kernel of $k \times k$, and we set to $k = 3$.

We can obtain the spatial weight coefficients M_T^s and M_H^s for the features T_{out} and H_{out} . M_T^s and M_H^s are multiplied with the input feature to obtain the fusion features. In addition, we also add the residuals to the fusion module. The specific formula is

$$\begin{aligned} T^s &= \text{Softmax}(M_T^s)T_{\text{out}} + T_{\text{out}} \\ H^s &= \text{Softmax}(M_H^s)H_{\text{out}} + H_{\text{out}}. \end{aligned} \quad (28)$$

The fusion features T^s and H^s of the two parts are output through the fully connected layer, and the final output Y is expressed as follows:

$$Y = \text{Softmax}(W(T^s \oplus H^s) + B) \quad (29)$$

where W and B represent the weight and bias of the output fully connected layer.

3) *Composite Loss Function*: For the network training, we add the prediction components from two subnetworks, PMCsN and MGCsN, in the loss function of AMGCFN to achieve branch balance. As the general solution to multiclassification problems, we use cross entropy to calculate the difference between the predicted value and the truth label value

$$\mathcal{L} = - \sum_{i=1}^N \sum_{j=1}^C L_j(p_i) \log(Y_j(p_i)) \quad (30)$$

where $L_j(p_i)$ represents the j th element of the label $L(p_i)$ and $Y_j(p_i)$ represents the probability that the pixel belongs to the j th class.

By (30), we can calculate the cross entropy according to the predicted values from AMGCFN, PMCsN, and MGCsN and add the cross entropy of the two subnetworks as auxiliary components to participate in backpropagation. The purpose is to make the two subnetworks for balance training

$$\mathcal{L} = \mathcal{L}_{\text{AMGCFN}} + \mathcal{L}_{\text{PMCsN}} + \mathcal{L}_{\text{MGCsN}}. \quad (31)$$

The entire model is trained in an end-to-end manner, and the full-batch gradient descent is used to update the network parameters of AMGCFN.

III. EXPERIMENTS

A. Dataset Description

In our experiments, three publicly available benchmark HSI datasets are used to evaluate the proposed AMGCFN: the Indian Pines, the University of Pavia, and the Salinas datasets.

1) *Indian Pines Dataset*: The first dataset is the Indian Pines dataset captured by an agricultural test site located in northwest Indiana. The size of the HSI is 145×145 pixels and contains 220 bands in total. After removing interfering bands, such as low signal-to-noise ratio and water vapor absorption, a total of 200 bands are remained. It has a spectral coverage of 0.4–2.5 μm , a spatial resolution of 20 m, and contains 10366 samples from 16 terrain categories; 2%, 2%, and 96% of samples per class are randomly selected for training, validation, and testing, respectively. The training and testing sets are shown in Table I.

TABLE I
CATEGORY INFORMATION OF THE INDIAN PINES DATASET

Class	Color	Class Name	Training	Validation	Test
1		Alfalfa	1	1	44
2		Corn-notill	29	29	1370
3		Corn-mintill	17	17	796
4		Corn	5	5	227
5		Grass-pasture	10	10	463
6		Grass-trees	15	15	700
7		Grass-pasture-mowed	1	1	26
8		Hay-windrowed	10	10	458
9		Oats	1	1	18
10		Soybean-notill	20	20	952
11		Soybean-mintill	50	50	2355
12		Soybean-clean	12	12	569
13		Wheat	5	5	195
14		Woods	26	26	1213
15		Building-Grass-Trees-Drives	8	8	370
16		Stone-Steel-Towers	2	2	89
Total			212	212	9845

TABLE II
CATEGORY INFORMATION OF THE UNIVERSITY OF PAVIA DATASET

Class	Color	Class Name	Training	Validation	Test
1		Asphalt	14	14	6555
2		Meadows	38	38	18573
3		Gravel	5	5	2089
4		Tress/Grass	7	7	3050
5		Metalsheets	3	3	1339
6		Baresoil	11	11	5007
7		Bitumen	3	3	1320
8		Bricks	8	8	3666
9		Shadows	2	2	938
Total			91	91	42573

2) *University of the Pavia Dataset*: The second dataset is the University of Pavia dataset obtained by the reflective optical system imaging spectrometer (ROSIS) sensor of the University of Pavia in northern Italy. The HSI size is 610×340 pixels and contains 115 bands. After removing the interference bands, the remaining 103 bands were used for research, with spectral coverage ranging from 0.43 to $0.86 \mu\text{m}$, a spatial resolution of 1.3 m , and a total of $42\,776$ samples of 9 categories were labeled; 0.2% , 0.2% , and 99.6% of samples from each class are randomly selected for training, validation, and testing, respectively. Details of this HSI are shown in Table II.

3) *Salinas Dataset*: The third dataset, Salinas, was collected by airborne visible infrared imaging spectrometer (AVIRIS) sensors over the Salinas Valley, CA, USA. The size of the dataset is 512×217 , including 224 bands, 20 water absorption bands are removed, and the remaining 204 bands are used for research. The Salinas dataset has a spatial resolution of 3.7 m and contains a total of $54\,129$ samples across 16 land-cover categories. Table III details the randomly selected 0.2% , 0.2% , and 99.6% samples for training, validation, and testing, respectively.

B. Experimental Setting

Our proposed AMGCFN consists of three modules: PMCsN, MGCsN, and MSAFF. The superpixel segmentation

TABLE III
CATEGORY INFORMATION OF THE SALINAS DATASET

Class	Color	Class Name	Training	Validation	Test
1		Brocoli gree weeds 1	5	5	1999
2		Brocoli gree weeds 2	8	8	3710
3		Fallow	4	4	1968
4		Fallow rough plow	3	3	1388
5		Fallow smooth	6	6	2666
6		Stubble	8	8	3943
7		Celery	8	8	3563
8		Grapse untrained	23	23	11225
9		Soli vinyard develop	13	13	6177
10		Corn senesced green weeds	7	7	3264
11		Lettuce romaine 4wk	3	3	1062
12		Lettuce romaine 5wk	4	4	1919
13		Lettuce romaine 6wk	2	2	912
14		Lettuce romaine 7wk	3	3	1064
15		Vinyard untrained	15	15	7238
16		Vinyard vertical trellis	4	4	1799
Total			116	116	53897

TABLE IV
ARCHITECTURE CONFIGURATION OF AMGCFN

Module	Layer	Kernel
Conv 7×7	CNN 1	$1 \times 1@64$
	CNN 2	$7 \times 7@64$
Conv 5×5	CNN 1	$1 \times 1@64$
	CNN 2	$5 \times 5@64$
Conv 3×3	CNN 1	$1 \times 1@64$
	CNN 2	$3 \times 3@64$
Graph Conv	GCN 1	@64
	GCN 2	@64

scale λ in MGCsN is fixed to 100. Other detailed configurations are summarized in Table IV. We use the Adam optimizer to train our network with a learning rate equal to 0.0005. The number of training iterations is 500. Our code is completed with Python 3.9 and Pytorch 1.5.1. The experimental environment consists of an i5-10500 CPU and a NVIDIA GeForce RTX 3090 GPU.

C. Comparison of Classification Performance

In this section, the three datasets are used to estimate the performance of AMGCFN. We compare with the following state-of-the-art methods: the support vector machine (SVM) with a radial basis function (RBF) kernel [45], the contextual deep CNN (CDCNN) based on 2-D CNN and ResNet [46], the spectral-spatial residual network (SSRN) based on 3-D CNN [47], the fast dense spectral-spatial convolutional network (FDSSC) [48], the DBDA mechanism network [22], the GCN [28], and CEGCN [34]. We also include results from two subnetworks for comparison: PMCsN and MGCsN. The subnetwork multipath results are fused by the MAFM module. All compared methods adopt the hyperparameters recommended in their original paper. The overall accuracy (OA), average accuracy (AA), and kappa coefficient (KC) are used as the evaluation indicators of classification performance. Each experiment is repeated ten

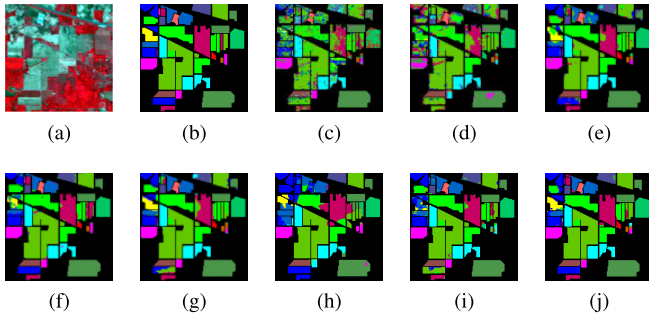


Fig. 7. Classification maps on the Indian Pines dataset. (a) False-color image. (b) Ground truth. (c) SVM (OA = 68.45%). (d) CDCNN (OA = 63.72%). (e) SSRN (OA = 89.86%). (f) FDSSC (OA = 93.44%). (g) DBDA (OA = 93.71%). (h) GCN (OA = 83.00%). (i) CEGCN (OA = 89.96%). (j) AMGCFN (OA = 95.65%).

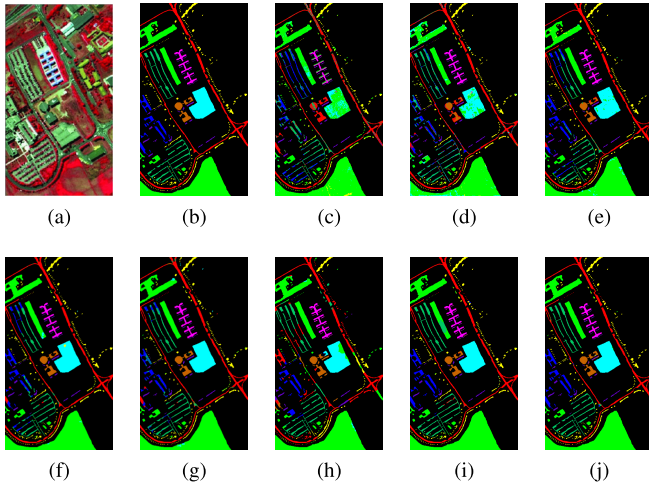


Fig. 8. Classification maps on the University of Pavia dataset. (a) False-color image. (b) Ground truth. (c) SVM (OA = 76.92%). (d) CDCNN (OA = 86.12%). (e) SSRN (OA = 93.67%). (f) FDSSC (OA = 95.74%). (g) DBDA (OA = 95.77%). (h) GCN (OA = 85.34%). (i) CEGCN (OA = 97.12%). (j) AMGCFN (OA = 98.99%).

times, in which the average accuracies are displayed in Tables V–VII. Also, the classification maps generated by these methods are illustrated in Figs. 7–9.

From the experimental results, limited by the small number of samples in this experiment, the models that rely on a large number of training samples, such as SVM and CDCNN, perform poorly on all three datasets. Taking the Indian Pines dataset as an example, on the seventh type of data, there are very few training samples, so SVM and CDCNN only obtain OA of 45.46% and 30.35%. Furthermore, CNN-based models with more complex structures, such as FDSSC and DBDA, achieve higher classification accuracy than GCN-based networks. This can also be seen from the subnetwork experiment results that the PMCsN has higher OA, AA, and KC than MGCsN. The reason for the above results is the complex distribution of ground objects in the Indian Pines dataset, and the ground object areas with the same label are small. Therefore, the pixel purity inside the superpixel block is low, which makes similar nodes have greater differences, which affects the features from GCN. In addition, due to the proportional selection of the training samples, there is a large gap in the number of samples of various types in the dataset. In the graph structure with fixed neighbors, it is difficult to effectively express the class-imbalanced node relationship.

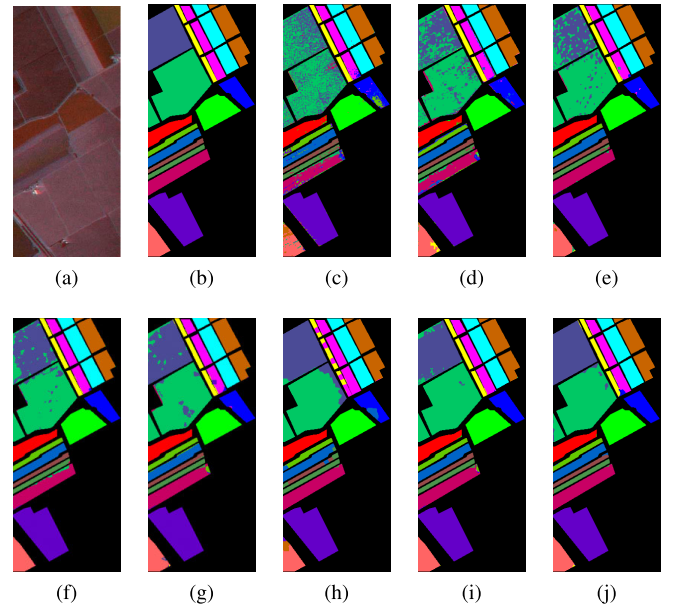


Fig. 9. Classification maps on the Salinas dataset. (a) False-color image. (b) Ground truth. (c) SVM (OA = 84.66%). (d) CDCNN (OA = 88.37%). (e) SSRN (OA = 93.70%). (f) FDSSC (OA = 96.15%). (g) DBDA (OA = 96.42%). (h) GCN (OA = 94.76%). (i) CEGCN (OA = 98.24%). (j) AMGCFN (OA = 98.87%).

This argument can be corroborated by the experimental results on the Salinas dataset. The regions of the same label in the Salinas dataset are more aggregated, so the interior of each superpixel obtained by SLIC segmentation is more pure, which is conducive to better expression of sample features. In addition, since the receptive field of CNN is fixed, it can only collect the information of neighbors around pixels. GCN has the ability to extract features from the global structure, which can effectively extract longer-distance relevant information. Therefore, on the Salinas dataset, GCN and CEGCN achieve the OA of 94.03% and 96.75%, outperforming any single CNN kernel algorithm. Again, the results of the subnetworks reflect that the MGCsN performs better than PMCsN, while achieving the highest accuracies for multiple categories. For the Salinas dataset, the distance between sample data with the same label is smaller, and the sample data with different labels are larger, which is also more suitable for GCN. The classification map obtained by the GCN model is smoother inside the same label results, while the classification map obtained by the CNN model has more heterogeneous labels mixed in the same land-cover areas.

Compared with the above two datasets, the dataset from the University of Pavia is more balanced. The image size is larger compared with Indian Pines, but the clusters of similar samples are smaller compared with Salinas. Therefore, on this dataset, the classification accuracy of the subnetwork MGCsN is better than that of PMCsN, but the gap between them is closer than that of the Salinas dataset. Hybrid networks are better at dealing with this situation, such as CEGCN, which achieves the OA of 95.88%, because the hybrid network combines the characteristics of CNN and GCN. It takes into account the ability of detailed description and long-range information extraction, so it can obtain more accurate classification map.

TABLE V
CLASSIFICATION RESULTS ON THE INDIAN PINES DATASET

Class	SVM	CDCNN	SSRN	FDSSC	DBDA	GCN	CEGCN	PMCsN	MGCsN	AMGCFN
1	30.40±15.57	16.27±15.68	86.71±13.30	67.96±41.31	88.25±29.60	38.19±46.78	57.04±5.33	76.95±17.89	95.46 ±0.10	90.40±8.30
2	52.52±1.34	56.45±7.83	79.37±8.58	87.90±4.94	86.79±7.19	65.46±7.34	90.57±3.58	90.46±2.61	81.29± 7.68	90.98 ±5.33
3	55.33±8.88	34.52±13.30	87.98±5.53	85.66±10.32	89.63±6.20	80.74±10.1	72.83±5.85	91.69±6.20	83.55±3.34	93.74 ±2.80
4	38.24±7.51	42.60±29.20	81.57±15.38	86.98±29.26	93.97 ±6.57	76.99±18.45	42.01±16.17	76.92±17.05	89.77±11.5	90.15±9.70
5	82.19±7.13	75.22±12.59	98.42 ±2.31	98.04±3.10	98.82±1.47	85.58±6.12	87.92±6.35	84.24±8.47	83.02±8.14	88.86±5.03
6	81.14±3.50	74.57±9.30	95.93±3.44	96.26±4.53	95.14±7.61	85.82±5.27	98.77 ±1.11	96.65±1.68	92.43±5.29	96.64±2.54
7	45.46±23.48	30.35±20.88	75.42±26.80	86.61±21.26	57.53±14.43	20.00±40.00	60.17±29.73	90.37±15.82	38.52±47.35	95.56 ±9.49
8	89.76±2.27	87.44±4.27	95.87±4.68	96.95±3.48	98.83±3.50	98.89±0.15	99.32±1.59	98.34±2.53	99.11±1.88	99.96 ±0.08
9	33.82±13.85	39.14±28.98	65.01±28.12	73.91±38.05	81.28±28.97	26.49±27.48	28.31±20.25	90.43 ± 15.75	62.85±29.96	89.47±15.25
10	58.07±5.78	49.21±7.71	82.02±5.99	85.36±12.92	85.90±6.76	71.93±5.76	87.44±6.11	89.12±5.55	79.54±6.85	89.66 ±5.74
11	58.32±5.26	57.40±4.01	84.75±4.37	90.17±6.57	89.67±6.05	85.58±6.27	96.12±1.57	94.09±3.09	94.87±2.89	96.35 ±3.04
12	41.67±7.68	35.95±7.60	83.77±10.52	87.23±6.13	86.06 ±13.53	73.18±9.65	64.72±14.72	77.72±9.69	75.51±7.23	81.42±7.01
13	84.59±6.05	80.50±12.45	97.80±2.87	97.55±3.88	96.99±4.13	94.26±14.55	98.87±1.91	98.26±2.22	99.03 ±1.02	97.48±6.36
14	85.36±3.36	78.93±7.90	93.48±3.31	93.85±2.82	95.19±4.82	96.59±4.68	99.71 ±0.29	97.44±1.87	97.68±3.80	99.15±1.05
15	60.00±9.48	61.70±8.22	90.16±5.39	95.51±4.83	90.64±9.66	65.95±28.65	71.67±1.08	95.50±2.96	95.34±9.31	98.21 ±2.72
16	98.41±2.39	74.77±11.54	85.12±28.68	97.12 ±2.88	90.42±10.75	14.07±3.11	79.36±16.10	82.73±16.07	88.86±16.32	92.21±7.98
OA(%)	63.90±2.33	61.07±2.60	87.03±3.39	89.87±2.94	90.46±5.35	80.51±1.72	88.32±1.31	91.84±1.31	89.01±1.71	93.94 ±1.12
AA(%)	62.21±2.47	55.94±5.31	86.46±5.32	89.19±4.71	89.07±8.07	67.55±5.71	73.97±2.93	89.43±2.60	84.80±5.16	93.14 ±1.10
KC(×100)	58.12±2.99	54.60±3.19	85.11±3.97	88.42±3.37	89.08±6.19	77.79±1.97	86.55±1.53	90.70± 1.50	87.49±1.95	93.09 ±1.27

TABLE VI
CLASSIFICATION RESULTS ON THE UNIVERSITY OF PAVIA DATASET

Class	SVM	CDCNN	SSRN	FDSSC	DBDA	GCN	CEGCN	PMCsN	MGCsN	AMGCFN
1	78.28±4.32	85.88±3.04	95.38±3.27	97.15±1.68	92.87±4.41	71.19±7.93	98.46 ±2.07	93.86±4.34	90.04±5.12	96.35±2.35
2	75.99±2.86	89.42±2.71	95.32±2.65	96.78±1.86	98.69±0.66	95.30±2.14	98.39±2.57	99.03±1.21	98.16±1.02	99.53 ±0.51
3	31.67±15.85	48.73±8.46	67.57±13.30	73.65±8.78	79.96±13.00	70.50±14.84	84.54±14.59	67.21±9.92	89.27±13.41	93.21 ±6.73
4	84.95±11.10	95.00±2.07	98.17±3.24	99.11 ±1.17	97.50±2.07	37.06±8.26	83.31±10.37	87.97±9.94	80.08±5.64	93.22±3.53
5	97.51±2.39	96.72±2.60	99.75±0.24	99.46±1.21	99.01±1.14	99.84±0.40	100.00 ±0.00	99.13±1.67	99.81±0.30	99.37±0.16
6	69.12±16.8	70.58±8.02	91.79±4.71	96.53±2.35	95.97±1.72	91.48±5.85	98.07±3.55	92.88±5.86	99.35 ±1.06	99.22±1.80
7	20.46±24.62	57.85±14.88	83.77±9.40	94.35±4.14	87.40±9.73	78.01±10.70	94.78±1.18	73.56±16.64	87.85±10.41	95.91 ±3.65
8	64.00±6.39	68.56±6.22	75.90±5.68	81.56±5.52	76.05±7.00	79.75±8.37	96.05±2.32	94.83±3.82	90.77±4.69	96.79 ±3.35
9	99.88±0.10	80.31±15.37	98.23 ±1.07	96.89±2.73	95.21±3.29	14.02±8.58	77.56±20.69	87.17±7.94	54.34±18.20	73.75±19.58
OA(%)	74.22±1.81	81.84±1.98	91.49±2.02	94.29±1.07	93.84±1.64	82.19±2.06	95.88±1.90	93.74±1.69	93.43±1.73	97.31 ±0.90
AA(%)	69.10±4.47	77.01±2.92	89.54±2.84	92.83±1.19	91.41±2.32	70.79±3.02	92.35±3.45	88.40±2.78	87.74±3.47	94.15 ±2.77
KC(×100)	63.95±2.75	75.75±2.69	88.67±2.73	92.39±1.45	91.83±2.19	76.27±2.71	94.54±2.50	91.66±2.24	91.26±2.31	96.43 ±1.19

Taken together, our AMGCFN outperforms all compared algorithms, including subnetworks on the three datasets. AMGCFN can show strong robustness and generality for different image sizes and different sample distributions. The feature extraction capability of AMGCFN benefits from the breakthrough of the multiscale limitation of a single receptive field in PMCsN and the capture of long-distance information in MGCsN. The structure of the AMGCFN hybrid network effectively fuses the features from the subnetworks, and the classification map is more accurate.

IV. DISCUSSION AND ANALYSIS

A. Ablation Analysis

Our AMGCFN consists of four parts: PMCsN, MGCsN, MAFM, and CAFM and uses the auxiliary loss function according to (31). To evaluate the effectiveness of each module, we design and conduct ablation experiments on three datasets. We compare the proposed AMGCFN with the two subnetworks (PMCsN and MGCsN), the elimination of auxiliary loss function, and the elimination of MAFM and CAFM. The experimental results are shown in Table VIII.

Experimental results on three datasets show that each component of the model contributes to improve performance. Specifically, by comparing (1) with (2) and (3) with (4), we can see the role of the MAFM module to improve the classification performance of the corresponding branch. It can be seen from (5) that the performance of the combination of two branches exceeds the effect of a single branch. However, no CAFM will make the training of the subnetwork relatively independent, and the promotion and improvement of the fusion of the CAFM part can be seen through the comparison of (5) and (7). The improvement of (7) compared with (6) illustrates the usefulness of the auxiliary loss, which makes the training of the subnetwork more balanced. Experiments demonstrate the effectiveness of each module, helping AMGCFN to obtain finer classification maps.

B. Influence of Segmentation Scale λ

The segmentation scale λ is directly related to the correspondence between pixels and superpixels when the graph is constructed by the SLIC algorithm. A larger λ means that more pixels correspond to a superpixel region,

TABLE VII
CLASSIFICATION RESULTS ON THE SALINAS DATASET

Class	SVM	CDCNN	SSRN	FDSSC	DBDA	GCN	CEGCN	PMCsN	MGCsN	AMGCFN
1	98.95±1.26	91.61±12.08	99.97±0.06	100.00±0.00	94.59±1.50	100.00±0.00	100.00±0.00	98.50±1.97	100.00±0.00	99.88±0.27
2	97.67±1.85	91.29±5.26	99.63±0.56	100.00±0.00	96.48±9.78	98.72±1.76	100.00±0.00	99.89±0.22	98.69±2.54	99.86±0.37
3	80.02±7.25	80.14±9.93	94.39±3.69	97.21±1.44	97.68±1.53	95.98±5.39	97.12±8.26	93.10±12.80	98.81±1.83	99.57±0.93
4	97.23±0.75	91.95±4.49	95.80±3.15	95.35±1.82	93.00±4.02	75.51±13.10	99.55±0.36	97.42±3.82	93.80±8.97	98.78±2.25
5	92.58±3.15	88.76±7.00	95.61±3.57	98.32±2.37	99.33±0.62	94.76±3.73	97.73±3.90	94.72±5.73	88.45±9.62	96.03±2.13
6	99.91±0.09	97.46±2.52	99.97±0.01	99.80±0.39	99.46±0.77	99.59±0.89	99.96±0.05	99.16±1.42	98.86±1.27	99.96±0.11
7	94.09±2.85	92.02±6.60	98.93±1.23	99.92±0.10	98.81±1.48	97.78±2.17	99.95±0.06	99.10±1.26	98.01±5.77	99.92±0.20
8	67.57±3.16	74.79±6.36	78.68±4.17	88.23±3.61	91.60±2.73	88.10±2.64	94.26±1.82	90.70±2.53	98.31±1.73	95.63±2.65
9	96.33±2.34	96.07±2.42	99.41±0.29	99.45±0.25	98.49±1.08	99.92±0.23	100.00±0.00	99.97±0.09	100.00±0.00	100.00±0.00
10	80.47±5.99	84.02±6.31	94.14±4.34	97.75±2.61	97.27±2.20	97.52±3.71	94.12±1.92	92.63±5.37	97.81±2.23	96.67±3.12
11	80.76±13.06	69.18±11.4	94.91±2.89	95.32±2.62	94.95±4.04	90.71±9.08	99.02±0.91	95.67±6.76	95.52±5.29	96.87±4.84
12	90.03±7.79	89.79±7.16	98.39±1.60	98.63±1.58	98.87±1.68	95.25±3.89	99.98±0.03	99.45±1.59	95.57±1.02	99.43±1.20
13	90.93±4.63	93.28±3.99	96.54±2.75	99.75±0.68	99.79±0.16	63.46±14.09	98.29±2.35	95.86±9.43	74.97±15.45	96.72±3.38
14	86.52±11.11	92.21±4.93	96.86±1.83	94.84±4.26	95.55±2.55	91.46±7.34	99.41±0.56	98.62±1.35	95.05±4.57	98.82±0.80
15	62.37±5.23	61.20±6.35	78.42±4.89	85.33±4.53	86.67±6.20	94.80±5.36	91.24±6.33	87.17±7.00	98.06±2.16	94.30±3.00
16	96.65±4.46	94.82±4.55	99.04±1.03	99.87±0.34	99.12±1.73	94.23±5.62	93.30±6.88	90.57±6.42	100.00±0.00	96.85±4.35
OA(%)	83.22±0.94	82.83±2.32	91.14±1.16	94.67±0.98	94.86±1.53	94.03±0.74	96.75±1.06	94.61±0.80	97.07±1.04	97.59±0.79
AA(%)	88.25±1.30	86.79±2.14	95.04±0.75	96.86±0.74	96.35±1.64	92.36±1.18	97.75±0.79	95.78±1.32	95.14±1.53	98.08±0.56
KC(×100)	81.26±1.06	80.88±2.59	90.12±1.29	94.06±1.09	94.28±1.70	93.36±0.82	96.38±1.18	93.99±0.89	96.73±1.16	97.32±0.86

TABLE VIII
OAS (%) OF ABLATION OF EACH MODULE ON THREE DATASETS

Module	(1)	(2)	(3)	(4)	(5)	(6)	(7)
PMCsN	✓	✓	-	-	✓	✓	✓
MGCsN	-	-	✓	✓	✓	✓	✓
MAFM	-	✓	-	✓	✓	✓	✓
CAFM	-	-	-	-	-	✓	✓
Auxiliary Loss	-	-	-	-	✓	-	✓
Indian Pines	91.72 ± 1.33	91.84 ± 1.31	88.19 ± 1.79	89.01 ± 1.71	93.08 ± 0.98	92.41 ± 1.75	93.94 ± 1.12
University of Pavia	92.86 ± 1.92	93.71 ± 1.69	91.69 ± 0.89	93.43 ± 1.73	96.29 ± 1.54	95.42 ± 1.30	97.31 ± 0.90
Salinas	93.59 ± 1.03	94.61 ± 0.80	96.86 ± 0.69	97.07 ± 1.04	97.14 ± 1.02	96.55 ± 1.04	97.59 ± 0.79

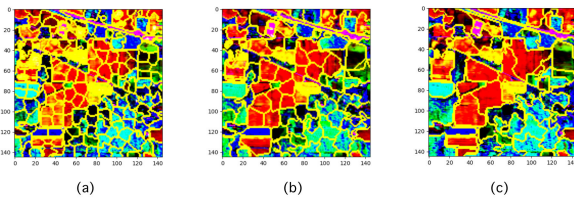


Fig. 10. Superpixel images with (a) $\lambda = 100$, (b) $\lambda = 200$, and (c) $\lambda = 500$.

so there are fewer graph nodes and a smaller graph structure. Fig. 10 shows the superpixel images at different segmentation scales. In our model, MGCsN is directly affected by superpixel segmentation, so we explore and experiment with it. We choose λ as 100, 200, 300, 400, and 500 to conduct experiments on three datasets. Each experiment was repeated ten times, and the average OA is shown in Fig. 11.

It can be seen that as the segmentation scale increases, the performance of MGCsN on the Indian Pines dataset drops most significantly compared with the other two datasets. This is due to the small size of the Indian Pines dataset, the relatively dense distribution of land covers, and the close proximity of different categories of land covers. Therefore, as the superpixel node expands, one node is more likely

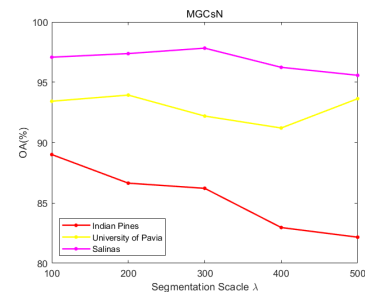


Fig. 11. OAs under different λ values on each dataset.

to be mixed with pixels of different categories. This will influence the classification results. The different types of land covers in the University of Pavia dataset are far apart, so the overall tends to be stable. In addition, as the superpixel node has more pixels, the interior of the similar land covers in the classification results of the Salinas dataset is smoother. Therefore, the classification performance increases slightly from $\lambda = 100$ to $\lambda = 300$. When λ is more than 400, each node with more pixels will result in the OA dropping.

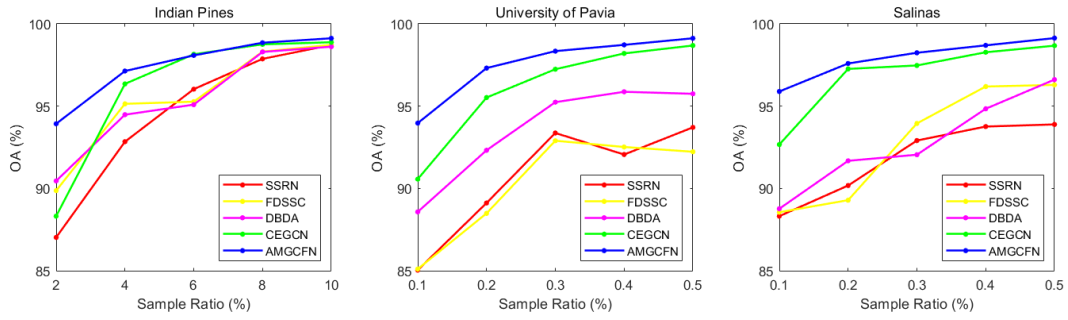


Fig. 12. Classification accuracy with different sample ratios.

TABLE IX
COMPARISON OF RUNNING EFFICIENCY FOR DIFFERENT METHODS

Data set	Training Time (s)			Params (K)
	Indian Pines	University of Pavia	Salinas	
CDCNN	66.2	27.6	30.3	1063.8
SSRN	79.2	36.1	42.2	364.2
FDSSC	134.7	42.9	66.5	1227.5
DBDA	219.0	47.9	114.0	382.4
CEGCN	6.7	31.3	32.0	166.4
AMGCFN	14.8	79.4	42.0	368.2

C. Comparison of Computational Efficiency

To evaluate the operational efficiency and computational consumption of our method, we compare different algorithms in terms of training time and number of network parameters (Params). The network parameters are computed on the Indian Pines dataset with 200 image bands and 16 categories. All methods are tested on the same computing platform described in Section III. Each experiment was repeated ten times to report the average time cost, and the experimental results are presented in Table IX. It can be seen that the training time and calculation consumption of our AMGCFN method are moderate within an acceptable range compared with the other networks.

D. Different Numbers of Training Samples

Pixel labeling of HSI is expensive, so training samples are often limited. In previous experiments, we have demonstrated that the AMGCFN has excellent performance under small samples. In this section, we test the performance of AMGCFN and four other models with different proportions of training samples on the three datasets. For the Indian Pines dataset, the sample proportions are 2%, 4%, 6%, 8%, and 10%, and for the University of Pavia and Salinas datasets, the sample proportions are 0.1%, 0.2%, 0.3%, 0.4%, and 0.5%. The OAs are shown in Fig. 12.

According to the experimental results, each algorithm shows better performance under the larger training sample ratio and under the maximum number of samples set in this experiment (Indian Pines 10%, and University of Pavia and Salinas 0.5%) to the best. AMGCFN always achieves the best performance. Relatively, as the number of samples decreases, the performance of each algorithm shows different degrees of decline. In particular, the performance is most obvious when the sample proportion decreases from 0.2% to 0.1%, and

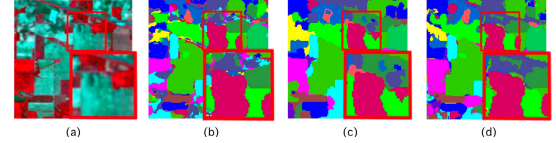


Fig. 13. Classification maps on the Indian Pines dataset. (a) False-color image. (b) PMCsN. (c) MGCsN. (d) AMGCFN.

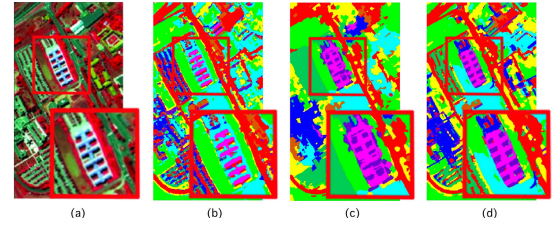


Fig. 14. Classification maps on the University of Pavia dataset. (a) False-color image. (b) PMCsN. (c) MGCsN. (d) AMGCFN.

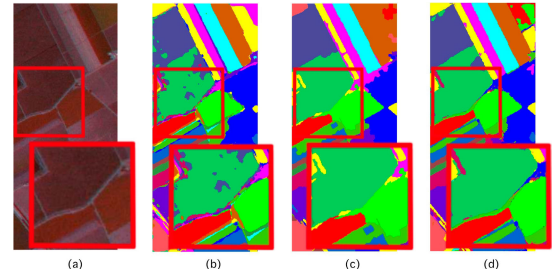


Fig. 15. Classification maps on the Salinas dataset. (a) False-color image. (b) PMCsN. (c) MGCsN. (d) AMGCFN.

the performance of CEGCN and FDSSC drops significantly. In contrast, our AMGCFN still has excellent performance when the sample size is small. This reflects the robustness of AMGCFN under different numbers of training samples.

E. Feature Fusion Analysis

The proposed AMGCFN model contains two subnetworks, PMCsN and MGCsN. We study the feature fusion effect of the subnetworks through their visual classification maps. Figs. 13–15 show all region classification maps for the two subnetworks and the entire network on the three datasets, and zoomed-in view to see the details. Taking the marked position of the red box as an example, we can observe that the CNN-based PMGCsN has a stronger ability to describe the classification details, especially the edge parts, and can correctly classify small areas, but will produce more salt and pepper noise in large areas. The classification map generated by GCN-based MGCsN is smooth inside the same-label regions, but it will cause some small regions to be

misclassified. After the fusion of the two subnetworks, the classification map takes into account the advantages of them. While paying attention to the edge features, the interior of the area is smoother, and the small area can also be correctly classified without a lot of salt and pepper noise. It can be said that the classification map of AMGCFN is closer to the ground truth than any single subnetwork, and the accuracy is higher.

V. CONCLUSION

In this article, we propose an attention fusion network based on multiscale convolution and multihop graph convolution to extract multilevel complex features for HSI classification, including two subnetworks (i.e., PMCsN and MGCsN) and two attention fusion modules (i.e., MAFM and CAFM). PMCsN is used to extract the multiscale features and MGCsN aims to learn the long-range information from multihop graph. MAFM adopts the multihead attention to fuse the multiscale and multihop features, and CAFM constructs a cross attention to implement the feature interaction of two subnetworks. The proposed AMGCFN can effectively aggregate the multilevel context features from the pixel-level and superpixel-level spatial-spectral information to improve the HSI classification task. Evaluation experiments on three benchmark HSI datasets show that AMGCFN has strong feature learning ability and feature fusion ability compared with the other advanced networks. In future work, we will focus on optimizing the network structure and promoting deeper integration of features to improve model training efficiency and classification accuracy.

REFERENCES

- [1] Y. Duan, F. Luo, M. Fu, Y. Niu, and X. Gong, "Classification via structure-preserved hypergraph convolution network for hyperspectral image," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, 2023, Art. no. 5507113.
- [2] Z. Wang, B. Du, L. Zhang, L. Zhang, and X. Jia, "A novel semisupervised active-learning algorithm for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 55, no. 6, pp. 3071–3083, Jun. 2017.
- [3] Z. Lv, F. Wang, G. Cui, J. A. Benediktsson, T. Lei, and W. Sun, "Spatial-spectral attention network guided with change magnitude image for land cover change detection using remote sensing images," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 4412712.
- [4] F. Luo, H. Huang, Z. Ma, and J. Liu, "Semisupervised sparse manifold discriminative analysis for feature extraction of hyperspectral images," *IEEE Trans. Geosci. Remote Sens.*, vol. 54, no. 10, pp. 6197–6211, Oct. 2016.
- [5] X. Li, L. Zhang, B. Du, L. Zhang, and Q. Shi, "Iterative reweighting heterogeneous transfer learning framework for supervised remote sensing image classification," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 10, no. 5, pp. 2022–2035, May 2017.
- [6] J. Devlin, M. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. North Amer. Chapter Assoc. Comput. Linguistics (NAACL)*, 2019, pp. 4171–4186.
- [7] M. Lan, J. Zhang, and Z. Wang, "Coherence-aware context aggregator for fast video object segmentation," *Pattern Recognit.*, vol. 136, pp. 1–12, Apr. 2023.
- [8] Z. Lv, H. Huang, L. Gao, J. A. Benediktsson, M. Zhao, and C. Shi, "Simple multiscale UNet for change detection with heterogeneous remote sensing images," *IEEE Geosci. Remote Sens. Lett.*, vol. 19, pp. 1–5, 2022.
- [9] F. Luo, T. Zhou, J. Liu, T. Guo, X. Gong, and J. Ren, "Multiscale diff-changed feature fusion network for hyperspectral image change detection," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, 2023, Art. no. 5502713.
- [10] H. Sun, X. Zheng, X. Lu, and S. Wu, "Spectral-spatial attention network for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 58, no. 5, pp. 3232–3245, May 2020.
- [11] Y. Chen, Z. Lin, X. Zhao, G. Wang, and Y. Gu, "Deep learning-based classification of hyperspectral data," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 7, no. 6, pp. 2094–2107, Jun. 2014.
- [12] W. Hu, Y. Huang, L. Wei, F. Zhang, and H. Li, "Deep convolutional neural networks for hyperspectral image classification," *J. Sensors*, vol. 2015, pp. 1–12, Jan. 2015.
- [13] Y. Chen, H. Jiang, C. Li, X. Jia, and P. Ghamisi, "Deep feature extraction and classification of hyperspectral images based on convolutional neural networks," *IEEE Trans. Geosci. Remote Sens.*, vol. 54, no. 10, pp. 6232–6251, Oct. 2016.
- [14] L. Ying, H. Zhang, and S. Qiang, "Spectral-spatial classification of hyperspectral imagery with 3D convolutional neural network," *Remote Sens.*, vol. 9, no. 1, p. 67, Jan. 2017.
- [15] E. Xie, N. Chen, J. Peng, W. Sun, Q. Du, and X. You, "Semantic and spatial-spectral feature fusion transformer network for the classification of hyperspectral image," *CAAI Trans. Intell. Technol.*, vol. 2023, pp. 1–15, Feb. 2023.
- [16] J. Yang, C. Wu, B. Du, and L. Zhang, "Enhanced multiscale feature fusion network for HSI classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 12, pp. 10328–10347, Jan. 2021.
- [17] X. Li, M. Ding, and A. Pižurica, "Deep feature fusion via two-stream convolutional neural network for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 58, no. 4, pp. 2615–2629, Apr. 2020.
- [18] S. Hao, W. Wang, Y. Ye, T. Nie, and L. Bruzzone, "Two-stream deep architecture for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 4, pp. 2349–2361, Apr. 2018.
- [19] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Int. Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 1–9.
- [20] G. Huang, Z. Liu, L. van der Maaten, and K. Q. Weinberger, "Densely connected convolutional networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Jun. 2017, pp. 4700–4708.
- [21] V. Ashish, N. Shazeer, and N. Parmar, "Attention is all you need," in *Proc. NeurIPS*, 2017, pp. 1–15.
- [22] R. Li, S. Zheng, C. Duan, Y. Yang, and X. Wang, "Classification of hyperspectral image based on double-branch dual-attention mechanism network," *Remote Sens.*, vol. 12, no. 3, p. 582, Feb. 2020.
- [23] Z. Xue, X. Yu, B. Liu, X. Tan, and X. Wei, "HResNetAM: Hierarchical residual network with attention mechanism for hyperspectral image classification," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 14, pp. 3566–3580, 2021.
- [24] Q. Liu, J. Peng, G. Zhang, W. Sun, and Q. Du, "Deep contrastive learning network for small-sample hyperspectral image classification," *J. Remote Sens.*, vol. 3, pp. 1–10, Jan. 2023.
- [25] L. Mou, P. Ghamisi, and X. X. Zhu, "Unsupervised spectral-spatial feature learning via deep residual Conv-Deconv network for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 1, pp. 391–406, Jan. 2018.
- [26] G. Camps-Valls, T. V. B. Marsheva, and D. Zhou, "Semi-supervised graph-based hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 45, no. 10, pp. 3044–3054, Oct. 2007.
- [27] F. Luo, Z. Zou, J. Liu, and Z. Lin, "Dimensionality reduction and classification of hyperspectral image via multistructure unified discriminative embedding," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5517916.
- [28] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proc. ICLR*, 2017, pp. 1–14.
- [29] L. Mou, X. Lu, X. Li, and X. X. Zhu, "Nonlocal graph convolutional networks for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 58, no. 12, pp. 8246–8257, Dec. 2020.
- [30] S. Wan, C. Gong, P. Zhong, B. Du, L. Zhang, and J. Yang, "Multiscale dynamic graph convolutional network for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 58, no. 5, pp. 3162–3177, May 2020.
- [31] D. Yao et al., "Deep hybrid: Multi-graph neural network collaboration for hyperspectral image classification," *Defence Technol.*, vol. 2022, pp. 1–13, Feb. 2022.
- [32] Y. Ding, X. Zhao, Z. Zhang, W. Cai, and N. Yang, "Multiscale graph sample and aggregate network with context-aware learning for hyperspectral image classification," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 14, pp. 4561–4572, 2021.

- [33] P. Velickovic, G. Cucurull, and A. Casanova, "Graph attention networks," in *Proc. ICLR*, 2018, p. 12.
- [34] Q. Liu, L. Xiao, J. Yang, and Z. Wei, "CNN-enhanced graph convolutional network with pixel- and superpixel-level feature fusion for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 10, pp. 8657–8671, Oct. 2021.
- [35] Y. Ding et al., "Multi-feature fusion: Graph neural network and CNN combining for hyperspectral image classification," *Neurocomputing*, vol. 501, pp. 246–257, Aug. 2022.
- [36] Y. Dong, Q. Liu, B. Du, and L. Zhang, "Weighted feature fusion of convolutional neural network and graph attention network for hyperspectral image classification," *IEEE Trans. Image Process.*, vol. 31, pp. 1559–1572, 2022.
- [37] H. Gao, Y. Yang, C. Li, L. Gao, and B. Zhang, "Multiscale residual network with mixed depthwise convolution for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 4, pp. 3396–3408, Apr. 2021.
- [38] Q. Liu, L. Xiao, J. Yang, and Z. Wei, "Multilevel superpixel structured graph U-Nets for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5516115.
- [39] S. Wan, C. Gong, P. Zhong, S. Pan, G. Li, and J. Yang, "Hyperspectral image classification with context-aware dynamic graph convolutional network," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 1, pp. 597–612, Jan. 2021.
- [40] H. Xue, X.-K. Sun, and W.-X. Sun, "Multi-hop hierarchical graph neural networks," in *Proc. IEEE Int. Conf. Big Data Smart Comput. (BigComp)*, Feb. 2020, pp. 82–89.
- [41] Q. Liu, L. Xiao, F. Liu, and J. Huan, "SSCDenseNet: A spectral-spatial convolutional dense network for hyperspectral image classification," *Acta Electronica Sinica*, vol. 48, no. 4, pp. 751–762, Nov. 2020.
- [42] D. K. Hammond, P. Vandergheynst, and R. Gribonval, "Wavelets on graphs via spectral graph theory," *Appl. Comput. Harmon. Anal.*, vol. 30, no. 2, pp. 129–150, Mar. 2011.
- [43] S. Woo, J. Park, and J. Y. Lee, "CBAM: Convolutional block attention module," in *Proc. ECCV*, 2018, pp. 3–19.
- [44] R. Hou, H. Chang, and B. Ma, "Cross attention network for few-shot classification," in *Proc. NeurIPS*, 2019, pp. 1–12.
- [45] F. Melgani and L. Bruzzone, "Classification of hyperspectral remote sensing images with support vector machines," *IEEE Trans. Geosci. Remote Sens.*, vol. 42, no. 8, pp. 1778–1790, Aug. 2004.
- [46] H. Lee and H. Kwon, "Going deeper with contextual CNN for hyperspectral image classification," *IEEE Trans. Image Process.*, vol. 26, no. 10, pp. 4843–4855, Oct. 2017.
- [47] Z. Zhong, J. Li, Z. Luo, and M. Chapman, "Spectral-spatial residual network for hyperspectral image classification: A 3-D deep learning framework," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 2, pp. 847–858, Feb. 2018.
- [48] W. Wang, D. Shuguang, J. Zhongmin, and S. Liujie, "A fast dense spectral-spatial convolution network framework for hyperspectral images classification," *Remote Sens.*, vol. 10, no. 7, p. 1068, Jul. 2018.



Hao Zhou received the B.S. degree in electronic information science and technology from Peking University, Beijing, China, in 2019, and the M.S. degree in signal processing from Nanyang Technological University, Singapore, in 2022.

His research interests include machine learning, deep learning, and remote sensing image processing.



Fulin Luo (Senior Member, IEEE) received the B.E. degree in mechanical engineering and automation from Southwest Petroleum University, Chengdu, China, in 2011, and the M.E. and Ph.D. degrees in instrument science and technology from Chongqing University, Chongqing, China, in 2013 and 2016, respectively.

He was an Associate Researcher and a Post-Doctoral Researcher with the State Key Laboratory of Information Engineering in Surveying, Mapping and Remote Sensing (LIESMARS), Wuhan University, Wuhan, China, from 2017 to 2021. He was a Research Fellow with Nanyang Technological University, Singapore, from 2020 to 2021. He has been a Professor with the College of Computer Science, Chongqing University, since 2022. His research interests include remote sensing processing, computer vision, and biomedical analysis.



Huiping Zhuang (Member, IEEE) received the B.S. and M.E. degrees from the South China University of Technology, Guangzhou, China, in 2014 and 2017, respectively, and the Ph.D. degree from the School of Electrical and Electronic Engineering, Nanyang Technological University, Singapore, in 2021.

He is currently an Associate Professor with the Shien-Ming Wu School of Intelligent Engineering, South China University of Technology. His research interests include signal processing and machine learning algorithms.



Zhenyu Weng (Member, IEEE) received the B.S. degree in computer science from Sun Yat-sen University, Guangzhou, China, in 2013, and the Ph.D. degree in computer applied technology from Peking University, Beijing, China, in 2020.

He is currently a Research Fellow with the School of Electrical and Electronic Engineering, Nanyang Technological University, Singapore. His research interests include computer vision, machine learning, and multimedia information retrieval.



Xiuwen Gong received the B.E. and M.E. degrees from Anhui Normal University, Wuhu, China, in 2011 and 2014, respectively. She is currently pursuing the Ph.D. degree in artificial intelligence (AI) with the Faculty of Engineering, The University of Sydney, Sydney, NSW, Australia.

Her research interests include machine learning, deep learning, and AI.



Zhiping Lin (Senior Member, IEEE) received the B.E. degree in control engineering from the South China Institute of Technology, Canton, China, in 1982, and the Ph.D. degree in information engineering from the University of Cambridge, Cambridge, U.K., in 1987.

He worked at DSO National Labs, Singapore, and Shantou University, Shantou, China. He was an IEEE Circuits and Systems Society Distinguished Lecturer from 2007 to 2008. Since 1999, he has been with the School of Electrical and Electronic Engineering, Nanyang Technological University, Singapore. His research interests include signal processing and machine learning.